

Mission C — Tests, logique & résultats

Document opérationnel des **tests** de Mission C (orchestrator-go) : comment ils sont organisés, la **logique** des seuils/gates du CI, et les **résultats mesurés** (couverture, benchmarks, failover, sécurité). Complète `MISSION_C_FONCTIONNEMENT.md` (le *quoi*) sur le *comment c'est prouvé*.

Aligné sur `.github/workflows/*.yaml`, `.ci/`, `docs/bench/baseline.json` et l'état mesuré du 2026-08-19 (stack dev : docker compose etcd + Patroni/Postgres Spilo, Go 1.26).

1. Vue d'ensemble — pourquoi cette organisation

Le cluster AMANE tient 5 promesses qu'il faut **prouver en permanence** :

Promesse	Preuve	Menace
Contrats B↔C / A↔C stables	<code>tests/</code> (module isolé) + <code>buf breaking</code>	renumérotation de champs, changement de RPC
HA failover < 5 s	superviseur + mesures failover	crash, partition réseau
Convergence CRDT multi-site	tests CRDT + propagation e2e	doublons, réordonnancements, trous de seq
Chemin d'écriture gated	fencing lease / leadership	écriture sur non-leader
Sécurité (TLS/mTLS, deps)	govulncheck + gosec	CVE dans les dépendances, anti-patterns

Organisation en **5 jobs CI + 1 contrat (proto-contract)**, chacun avec un **rôle unique** et un coût maîtrisé (gating par chemins → pas de minutes brûlées sur des PR docs) :

Workflow	Rôle	Déclencheur
<code>proto-contract.yaml</code>	lint + breaking buf + verrou machine des numéros de champs	push main + PR
<code>build-go.yaml</code>	compilation, vet, tests unitaires, couverture (seuil + plancher)	push main + PR
<code>integration-tests.yaml</code>	contrats vs etcd réel via testcontainers	push main + PR
<code>chaos-tests.yaml</code>	scénarios de panne (crash, partition, fencing, CRDT chaotique)	PR touchant consensus/ , replication/ , supervisor/
<code>benchmark.yaml</code>	micro-benchmarks (PR chemins chauds) + macro failover (cron + dispatch)	PR chemins chauds + cron 02:15
<code>security-scan.yaml</code>	govulncheck + gosec, bloquant sur critiques	push main + PR + cron 02:30

Logique centrale : les tests d'intégration exigent un **etcd réel** (`AMANE_TEST_ETCD`) ; ils sont *skippés* sans lui. Deux jobs séparés résolvent ce problème sans polluer le build : `integration-tests/chaos-tests` *démarrent etcd via testcontainers* (module isolé `.ci/testcontainers`, même image qu'en dev `quay.io/coreos/etcd:v3.5.5`) et *exécutent la commande passée après -- avec la variable pointée dessus*. `build-go` *démarre etcd aussi** (sinon la couverture des intégrations ne serait jamais comptée).

2. La politique de couverture (option C)

Décision reposant sur la mesure : un seuil **80 %** était hors d'atteinte (68,7 % hors `gen/`). Option retenue : **seuil agrégé + plancher par paquet**, remonté par paliers.

Paramètre	Valeur	Pourquoi
MIN_COVERAGE (agrégat, gen/ exclu)	70 %	atteint (72,1 %) avec une marge volontairement faible
PKG_MIN_COVERAGE (plancher par paquet)	60 %	anti-gaming : un paquet à 40 % n'est jamais compensé par grpcserver à 85 %
Périmètre couvert	packages produit, cmd/ inclus, gen/ exclu	on n'exige pas de taux sur du code généré par buf
Plan de remontée	70 → 75 → 80 %	chaque PR de tests sur consensus/membership monte d'un palier

Garde-fous : etcd réel démarré **avant** la mesure (les intégrations comptent, ne sont pas skippees) ; -covermode=atomic ; la gate est rejouable à l'identique (go tool cover -func sur le profil filtré).

3. Organisation du code de test

orchestrator-go/ tests unitaires DANS chaque paquet (+ integration etcd si requis) consensus/ leadership_test (fencing), membership_integration (enrôlement/révocation/quorum) replication/ counter_test, journal_test, relay_test, propagator_test, bench_test supervisor/ evaluator_test (détecteur), patroni_test, config_test supervisor_integration (crash + garde anti-partition, FAUX Patroni) grpcserver/ server_test, transport_test, credentials_test (mTLS + TLS 1.3) mesh/ mesh_test, runtime_test, runtime_etcd_test (KV réel) telemetry/ tracecontext_test (parse/round-trip/racine/LogAttrs/StartSpan) tests/ MODULE ISOLÉ github.com/amane/tests – les contrats inter-missions contracts/ contracts_test.go (A↔C enroll/revoke, write gated, PushDelta) proto_contract_test.go (verrou numéros de champs + liste RPC) .ci/ outillage CI (hors modules produit) testcontainers/ module Go isolé : démarre etcd, propage AMANE_TEST_ETCD, re-exécute la commande benchcheck.sh médiane des ns/op vs baseline.json docs/bench/baseline.json baselines micro-benchmarks + SLO failover

Règle : un test qui exige etcd est marqué t.Skip sans AMANE_TEST_ETCD ; les unitaires restent verts hors etcd. Toujours -count=1 (pas de cache trompeur) et -race sur les paquets concurrents (replication, mesh, supervisor).

4. Résultats mesurés

4.1 Couverture (etcd réel compté, gen/ exclu)

Paquet	Avant PR tests	Après	Objectif 80 %
replication	95,7 %	95,7 %	■
grpcserver	85,3 %	85,3 %	■
supervisor	78,2 %	80,2 %	■
mesh	76,5 %	83,2 %	■
telemetry	70,5 %	93,2 %	■
consensus	66,4 %	73,8 %	■ +6,2 pt
Agrégat produit	68,7 %	72,1 %	■ +7,9 pt (seuil 70 ■, plancher 60 ■)

Tests ajoutés pour franchir le seuil (cibles qui étaient à 0 %) :

Fichier ajouté	Fonctions couvertes
telemetry/tracecontext_test.go (2 tests)	LogAttrs, StartSpan (nil-safe), branche isHex false

mesh/runtime_etcd_test.go	EtcdKV.Put, EtcdKV.List (round-trip réel)
supervisor/config_test.go	isPrimaryRole, Config.Validate (erreurs) + Defaults
consensus/waitctx_test.go + ajout intégration	waitCtx (annulé/timed-out), Registry.Quorum

4.2 Superviseur / failover (stack réelle, SLO V2 < 5 s)

Scénario	failover	writable	détection	Fencing	Zéro perte
Switchover planifié	~2,2 s	~2,4 s	—	OK	OK
Crash SIGKILL (superviseur actif)	3,2 s	4,0 s	0,8 s	OK	OK
Partition (coupure totale)	3,0 s	3,4 s	0,7 s	OK	OK
Crash sans superviseur	~21 s	~22 s	plancher ttl ≥ 20	OK	OK

Les scripts scripts/{switchover, failover}_measure.sh sont rejouables et auto-vérifiants (fencing + zéro perte), utilisés par le job benchmark (macro, cron + dispatch).

4.3 Micro-benchmarks (baseline, docs/bench/baseline.json)

Benchmark	baseline (ns/op, i5-6300U)	médiane rejouée	seuil CI (×10)
BenchmarkRelayAdd	1 214	1 242	12 140
BenchmarkRelayAccept	1 426	938	14 260
BenchmarkJournalWrite	1 016	1 259	10 160

Tolérance volontairement lâche (×10) : à l'échelle µs, le bruit machine/CI domine ; le vrai SLO (failover < 5 s) est porté par la mesure macro, pas par le µbench.

4.4 Sécurité

Outil	Version	Résultat
govulncheck	v1.7.0 (épinglée)	0 vulnérabilité appelée (1 vuln en module importé non appelé, non bloquante)
gosec	v2.28.0 (épinglée)	0 issue niveau high/high (gen/ exclu)

4.5 Chaos (etcd réel, exécuté via testcontainers)

Scénario	Test	Résultat
Failover sur crash	TestSupervisorAgainstEtcd (faux Patroni + vrai etcd)	ok (~10 s)
Garde anti-partition	TestSupervisorPartitionGuardAgainstEtcd	ok (jamais de forçage si heartbeat frais)
Élection / fencing	TestLeadershipFencingAgainstEtcd	ok
Convergence CRDT	go test ./replication/ -race	ok (rejeu, doublons, trous de seq, réflexion ignorée)

5. Comment lancer (reproductible)

```
# Suite complète locale (nécessite la stack dev : docker compose up -d) cd orchestrator-go &&
AMANE_TEST_ETCD=127.0.0.1:2379 go test ./... -count=1 -race # Contrats inter-missions (module isolé) cd
tests && AMANE_TEST_ETCD=127.0.0.1:2379 go test ./... -count=1 -race # Chaos – via le harness
testcontainers (etcd réel, sans compose local) cd .ci/testcontainers && go run . -workdir
../../orchestrator-go -- go test ./supervisor/ \ -run
'TestSupervisorAgainstEtcd|TestSupervisorPartitionGuardAgainstEtcd' -v -count=1 # Couverture à
l'identique du job CI cd orchestrator-go && AMANE_TEST_ETCD=127.0.0.1:2379 go test ./...
-coverprofile=coverage.out -covermode=atomic grep -v 'gen/amane/' coverage.out > cov-prod.out && go
tool cover -func=cov-prod.out | tail -1 # Micro-benchmarks + seuil bash .ci/benchcheck.sh # Mesures HA
bash scripts/switchover_measure.sh bash scripts/failover_measure.sh crash bash
scripts/failover_measure.sh partition # Sécurité cd orchestrator-go && go run
golang.org/x/vuln/cmd/govulncheck@v1.7.0 ./... cd orchestrator-go && go run
github.com/securego/gosec/v2/cmd/gosec@v2.28.0 \ -severity high -confidence high -exclude-dir=gen ./...
```

6. Points de vigilance

- **benchmark / macro-failover n'a jamais tourné sur un runner GitHub** : pull de l'image spilo (~1,5 Go), **docker kill/network disconnect, switchover_measure.sh** enchaîné — le chemin est identique à la stack locale, mais premier run CI à surveiller.
- **Marge de couverture étroite** : 72,1 % pour un seuil 70 % (2,1 pt). Un test d'intégration flaky qui se *skippe* en CI ferait repasser le gate au rouge. Viser 75 % après avoir couvert les chemins lease perdue / keepalive rompu / membership (consensus).
- **cmd/ (mains) à 0 %** : inclus dans l'agrégat (c'est lui qui tire 72 % vers le bas) mais pas de plancher spécifique — assumé, le code utile est dans les packages.
- **La CI ne s'activera qu'au premier push/PR** sur la branche par défaut du dépôt GitHub réel (les workflows sont créés mais non exécutés ici).